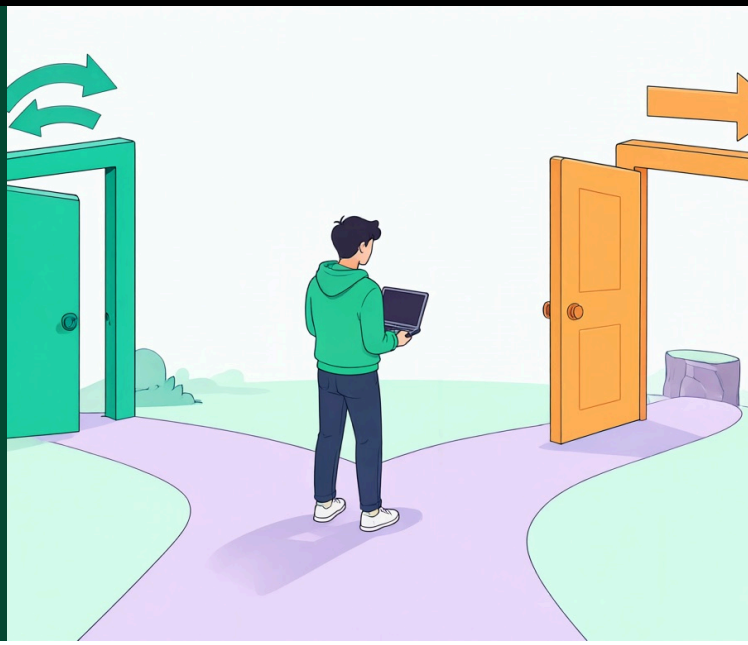
Problem and solution, simplified

Rollback restores the code. It can't restore what the code already did.

One-Way Door is a GitHub Copilot skill that spots, before merge, the changes you can't take back, and tells you how to ship them safely.



The problem, in one paragraph

Teams plan for a bad release by preparing to roll it back. That works for code. But some releases do things that stay done: a payment sent, a table rewritten, a message delivered to a partner. Rolling back the code doesn't undo them. Nobody checks, before merge, which changes are like that, so teams discover it in production, when it's too late.

What makes One-Way Door different

It asks a question nobody else asks

Not “does the new code work?” but “if it doesn't, can we go back, and what can't be undone?”

Before merge, on every pull request

It reads the diff while the change is still just code: the cheapest moment to fix it.

It names the point of no return

For each risky change: when the door closes, what breaks after that, and how long rollback stays safe.

It gives a safe way to ship

A step-by-step release plan where every step can be undone, and a bug shows up before it can do harm.

Where it runs

Inside GitHub Copilot: VS Code, the Copilot CLI, the cloud agent and code review. One folder in the repository; no servers, no accounts, no network calls.

In one line

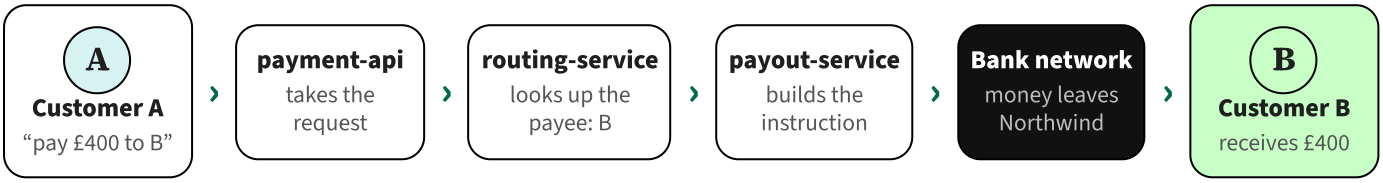
Know before you merge whether you can roll back, and what you can't.

A SIMPLE SCENARIO

A payment meant for B goes to C

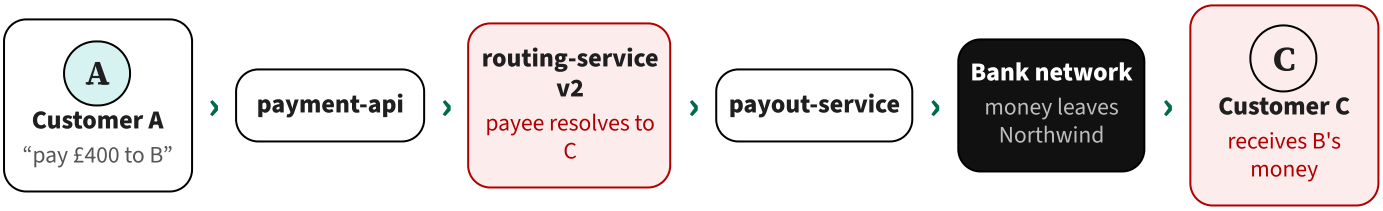
Northwind Payments moves money between accounts with a few small services. Customer A pays customer B every day, and it works.

How a payment travels (version 1)



Then version 2 is deployed

A tidy-up of routing-service changes how the payee is looked up. Tests pass, stage is green, the release checklist is signed off. But the new lookup reads the wrong account ID, so A's payments now resolve to customer C.



What happened, minute by minute (illustrative)

Time	What happens	Can it be undone?
09:00	Version 2 of routing-service is deployed.	**Yes: it's only code.**
09:02	The first payment from A goes to C. The bank network accepts it.	**No: the door has closed.**
09:02–09:40	120 payments, £48,300 in total, go to the wrong customer. Nothing errors.	**No: each one is final.**
09:40	Customer B calls: "Where's my money?"	—
09:45	On-call rolls back to version 1. New payments go to B again.	**The code: yes, in 5 minutes.**
09:45 →	£48,300 sits in C's account. Recall requests, customer complaints, incident report.	**The money: maybe, in days, not guaranteed.**

The rollback worked perfectly. It fixed the code in five minutes. It couldn't bring back a single payment.

WHAT DEVOPS AND INFRASTRUCTURE TEAMS ALREADY DO

Strong safety nets, all aimed at the code and the platform

Northwind's release process is good. Every one of these ran before version 2 went live, and every one passed.

Backups and disaster recovery

Protects: databases and infrastructure can be restored.

Doesn't cover: a payment the bank has already moved.

Production readiness on stage

Protects: the release deploys and runs in a prod-like stage.

Doesn't cover: stage pays test accounts; the bug looked like a working payment.

Regression and smoke tests

Protects: existing features still work after the change.

Doesn't cover: tests that assert “a payment was sent”, not “to the right person”.

Rollback exercises

Protects: the team can redeploy the previous version quickly.

Doesn't cover: what the bad version did while it was live.

Canary and blue-green deploys

Protects: a smaller share of traffic is exposed at first.

Doesn't cover: a canary still sends real payments: fewer, but just as final.

Monitoring and alerts

Protects: errors, latency and failed payments are noticed.

Doesn't cover: a payment that succeeds, to the wrong account, raises no error.

The gap they share

- They test the **new code going forward**. A rollback drill tests **going back**, but only for the code.
- They work at the level of the **environment and the release**, not the individual change.
- None of them asks: **does this change do something we can't take back?**
- So the answer arrives in production, after the money has moved.

Rollback restores the code. It can't restore what the code already did.

Payments sent, messages delivered to partners, data rewritten in place. These are one-way doors.

HOW ONE-WAY DOOR IS DIFFERENT

It looks at each change for what can't be undone

It doesn't replace the release process. It adds the one question the process never asks, at the one moment it's still cheap to act on.

	Today's safety nets	One-Way Door
The question	Does the new version work? Can we redeploy the old one?	**If this goes wrong, can we go back, and what can't be undone?**
When	After merge: on stage, during the release, in production	**Before merge, on the pull request**
Looks at	The running system and its environment	**The change itself: the diff, and who else reads its data**
Finds	Errors, failures, broken features	**Side effects that stay done: payments, partner messages, rewritten data**
Output	Pass or fail, an alert, a runbook	**A verdict, the point of no return, and a safe way to ship**
Who acts	Release, platform and on-call teams	**The author and reviewer, before anyone deploys**

What the skill does on every pull request

1. **Classify:** each change is a two-way door (safe to roll back) or a one-way door.
2. **Time it:** when the door closes, and what breaks or stays done after that.
3. **Blast radius:** other services and jobs that read the same data, even outside the PR.
4. **Split it:** a release in steps, each reversible, so a mistake surfaces before it's final.

What it prevents

- Money, messages or data that leave or change for good because of a bug.
- Rollbacks that fail, or make things worse, mid-incident.
- Hand-written fixes under pressure, and quiet data corruption.
- Surprises for teams outside the PR, such as finance or partners.

What it doesn't claim

- It doesn't replace tests, stage or rollback drills. It complements them.
- It doesn't promise to spot every logic bug.
- It makes sure a change that can't be undone ships in a way where a bug is caught **before** it's final.

THE SAME SCENARIO, WITH ONE-WAY DOOR

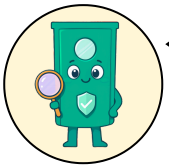
The bug is caught while it's still only a log line

GitHub Copilot reviewed the routing-service PR · used `one-way-door`

Verdict: ONE-WAY · Door closes: **the first payment instruction sent to the bank network** · Safe rollback window: **none**

1. **This changes who receives an outbound payment.** Once the bank accepts an instruction, it can't be recalled by redeploying.
2. **A rollback fixes future payments only.** Any payment sent while version 2 runs stays sent.
3. **Blast radius:** payout-service, the ledger, reconciliation and customer notifications all trust the payee this service returns.

Suggested split: run the new lookup in shadow first, compare, then switch with a guard. Reply “generate the split”.



This change moves money. Let's make sure a mistake shows up before any of it leaves.

Why it flags this: the skill's **External** rule says the door closes as soon as someone outside your deploy pipeline sees the change, and you can't roll back a partner. A bank network is that partner.

The rollback-safe way to ship it

1 Shadow

Version 2 works out the payee next to version 1. Version 1 still decides who gets paid.

IN THE SCENARIO

Every payment goes to B as before. v2 logs “C”.

2 Compare

Any payment where v1 and v2 disagree is a mismatch, and stops the rollout.

IN THE SCENARIO

100% of A's payments mismatch. Caught. £0 moved.

3 Guarded switch

Only after zero mismatches: v2 decides, but holds any payment whose payee changed.

IN THE SCENARIO

Never reached until the lookup is fixed.

4 Remove the old

Once v2 has run cleanly, delete v1’s lookup.

IN THE SCENARIO

Safe: nothing depends on it any more.

	Without the skill	With the skill
When the bug shows	**09:40, when customer B calls**	**In shadow, before the switch**
Money sent to C	**£48,300, 120 payments**	**£0**
Rollback needed	Yes, plus recalls and an incident	**No: fix the lookup and run shadow again**
Customers affected	**A, B and C**	**None**